

Reflection Log

Credit 2

Noorinder

Assignment Name: FindAndReplace

How has your program changed from planning to coding to now?

At first, the program was planned to prompt the user for a file name, search for a word or phrase within the file, and replace it with a new word or phrase specified by the user. The idea seemed straightforward: read the file content, perform the replacements, and overwrite the file. However, during the coding process, I encountered multiple challenges that required adjustments to ensure the program worked as expected.

1. File Reading Compatibility

- **Problem:** The initial implementation used `Files.readString()` to read the file content. This method caused issues with files containing unsupported encoding or large amounts of data.
- **Fix:** I switched to using `Files.readAllBytes()` and explicitly converted the bytes into a string. This improved compatibility and allowed the program to handle a variety of file types more reliably.

Before Code:

```
String content = Files.readString(Paths.get(filename)); // Unsupported encoding or large files caused errors
```

After Fix:

```
String content = new String(Files.readAllBytes(Paths.get(filename))); // More reliable file reading
```

2. Case Sensitivity in Search

- **Problem:** The program initially performed case-sensitive searches. If the user searched for "Hello," it wouldn't match "hello" or "HELLO." This caused the replacement to miss variations of the word.

- **Fix:** I modified the `replace()` call to use `replaceAll()` with a case-insensitive regex `((?i))`, ensuring all variations of the search term were replaced.

Before Code:

```
String updatedContent = content.replace(searchWord, replacementWord); // Case-sensitive
```

After Fix:

```
String updatedContent = content.replaceAll("(?i)" + searchWord, replacementWord); // Case-insensitive
```

3. File Overwriting Without Confirmation

- **Problem:** The program directly overwrote the file after replacing the content, which risked losing the original data if the replacement wasn't what the user intended.
- **Fix:** I added a confirmation prompt to allow the user to decide whether to overwrite the file or save the updated content to a new file.

Improved Code Example:

```
System.out.print("Would you like to overwrite the original file? (yes/no): ");
String confirmation = scanner.nextLine().toLowerCase();
if (confirmation.equals("yes")) {
    try (PrintWriter writer = new PrintWriter(filename)) {
        writer.write(updatedContent);
    }
    System.out.println("File overwritten successfully.");
} else {
    System.out.print("Enter a new file name to save the updated content: ");
    String newFileName = scanner.nextLine();
    try (PrintWriter writer = new PrintWriter(newFileName)) {
        writer.write(updatedContent);
    }
    System.out.println("Updated content saved to: " + newFileName);
}
```

4. Better Error Handling

- **Problem:** The program originally provided generic error messages like "File not found" or "An error occurred." This made it difficult for users to understand the specific issue.
- **Fix:** I updated the error messages to include details about the missing file or the type of I/O error encountered, providing more clarity.

Before Code:

```
System.out.println("An error occurred.");
```

After Fix:

```
catch (FileNotFoundException e) {
    System.out.println("The file '" + filename + "' does not exist. Please check the file name and try again.");
}
catch (IOException e) {
    System.out.println("An error occurred while reading or writing the file. Details: " + e.getMessage());
}
```

5. Preventing Replacement in Empty Files

- **Problem:** If the file was empty, the program still attempted to perform replacements, resulting in unnecessary writes.
- **Fix:** I added a check to ensure the file content was not empty before performing the replacement.

Improved Code Example:

```
if (content.isEmpty()) {
    System.out.println("The file is empty. No replacements can be made.");
} else {
    String updatedContent = content.replaceAll("(?i)" + searchWord, replacementWord);
    // Continue with the replacement logic
}
```